Smart Vial Backend System

Project Delivery Report

Backend Development Team

February 2026

Executive Summary

This document outlines what has been built for your Smart Vial medication tracking system. All requested features have been successfully implemented and are ready for your mobile app team to integrate.

Contents

1 Project Overview

The Smart Vial Backend is a cloud-based system that connects your ESP32 smart medication bottles to mobile applications. It tracks when patients open their medication bottles, monitors battery levels, and provides data to both patients and caregivers.

1.1 What Was Requested

You asked for a backend system that:

- Receives data from multiple smart bottles (ESP32 devices)
- Tracks medication adherence (when bottles are opened/closed)
- Manages multiple devices per user
- Provides caregiver access to patient data
- Ensures secure data access
- Works reliably even with unstable Wi-Fi

1.2 What Was Delivered

A complete, production-ready backend system with:

- Secure device communication system
- Patient and caregiver dashboards (API-ready)
- Multi-device support per user
- Comprehensive documentation
- Ready for mobile app integration

2 Core Features Delivered

2.1 Device Registry System

What it does: Keeps track of all your smart bottles.

What's stored for each device:

- Unique device ID (like a serial number)
- Who owns the device (patient)
- Last time the device connected
- Battery percentage
- Firmware version
- Whether it's currently active
- Assigned caregiver (if any)

Key capabilities:

- One patient can own multiple bottles
- Easy device claiming through QR code or manual entry
- Track device health (battery, connectivity)

2.2 Event Tracking System

What it does: Records every time a bottle is opened or closed.

What's recorded:

- Which bottle was used
- What happened (opened or closed)
- When it happened (device time and server time)
- Additional data (battery level at that moment)

Special features:

- **Duplicate prevention:** If the device sends the same event twice (bad Wi-Fi), it's only recorded once
- **Time accuracy:** Both device time and server time are recorded for accuracy
- **Complete history:** All events are saved permanently for adherence tracking

2.3 User Management

What it does: Manages patients and caregivers.

User types supported:

1. **Patients:** Own devices, can view their own data
2. **Caregivers:** Can view data from assigned patient devices
3. **Combined:** One person can be both (e.g., a parent caring for another family member)

Key capabilities:

- Secure login with JWT tokens
- Automatic user creation on first login
- Device claiming by patients
- Caregiver assignment by patients

2.4 Security Implementation

How we keep data secure:

1. **Device Security:**
   - Devices use a special API key to send data
   - Only registered devices can upload information
   - Prevents unauthorized devices from accessing the system
2. **User Security:**
   - JWT (JSON Web Token) authentication for mobile apps
   - Tokens expire after 30 days (configurable)
   - Each user can only see their own data
   - Caregivers can only see devices they're assigned to
3. **Data Privacy:**
   - Patients control who can see their data
   - Caregivers can be added or removed anytime
   - No data is visible without proper authorization

3 API Endpoints (For Mobile App)

Your mobile app team can now connect to the backend using these ready-to-use APIs:

3.1 For Patients

View All My Bottles
Shows all devices owned by the patient with battery status and last connection time.

Claim a New Bottle
Add a new smart bottle to the patient's account using device ID or QR code.

View Medication History
See when bottles were opened/closed, track adherence over time.

Assign a Caregiver
Give a family member or healthcare provider access to view your data.

Remove a Caregiver
Revoke caregiver access anytime.

3.2 For Caregivers

View Assigned Devices
See all bottles the caregiver has been given access to.

View Patient Adherence
Check when patients took their medication, view history.

Monitor Device Health
See battery levels and connection status for all assigned devices.

4 Firmware Integration Support

4.1 What Was Reviewed

We reviewed and verified the device (ESP32) communication to ensure:

- **Secure Connection:** Devices use HTTPS to send data
- **Authentication:** API key system implemented for device access
- **Retry Logic:** If Wi-Fi drops, devices will retry sending data
- **Time Synchronization:** Both device and server timestamps recorded
- **Battery Reporting:** Devices report battery level with every event

4.2 What Devices Send

Your ESP32 devices can now send:

1. **Events:** When bottle opens/closes
2. **Telemetry:** Battery percentage, connection status
3. **Timestamps:** When events occurred

4.3 Reliability Features

- Events buffered on device if offline
- Automatic retry if server doesn't respond
- Duplicate event prevention
- Time sync validation

5 Database Structure

5.1 What Data is Stored

Three main collections:

1. **Users:**
    - User ID, roles (patient/caregiver)
    - Owned devices, caregiving devices
    - Account creation and last login time
2. **Devices:**
    - Device ID, name, owner
    - Battery level, firmware version
    - Last seen, online status
    - Assigned caregiver
3. **Events:**
    - Event type (open/close)
    - Device timestamp and server timestamp
    - Device ID, idempotency key
    - Additional payload data

5.2 Database Technology

- **Technology:** MongoDB (flexible, scalable)
- **Hosting:** MongoDB Atlas (cloud-hosted, automatic backups)
- **Security:** Encrypted connections, access controls

6 Documentation Provided

Your development team has access to comprehensive documentation:

6.1 API Reference

- Complete list of all available endpoints
- Request/response examples for each API
- Authentication instructions
- Error handling guide
- Sample code (curl, Postman)

6.2 Architecture Documentation

- System design overview
- Data flow diagrams
- Security implementation details
- Scalability considerations

6.3 Database Schema

- Complete data models
- Field descriptions
- Relationships between collections
- Index strategies for performance

6.4 Development Guide

- How to run the system locally
- How to test APIs
- How to generate test tokens
- Environment setup instructions

6.5 Deployment Guide

- Production deployment checklist
- Environment variable configuration
- Security best practices
- Monitoring recommendations

6.6 Troubleshooting Guide

- Common issues and solutions
- Debugging tips
- FAQ section

7 What's NOT Included (As Requested)

To keep the system simple and maintainable, we explicitly did NOT build:

- Over-the-air (OTA) firmware updates
- Device remote control capabilities
- MQTT message brokers

- Complex IoT platforms (AWS IoT Core, Azure IoT Hub)
- Fleet management dashboards
- Cellular connectivity
- Real-time streaming

These features can be added later if needed, but keeping them out now makes the system easier to maintain and understand.

8 Technology Stack

8.1 Backend Framework

- **Node.js + Express:** Fast, widely-used, easy to maintain
- **Why:** Large developer community, excellent documentation

8.2 Database

- **MongoDB:** Flexible document database
- **Why:** Easy to scale, handles time-series data well

8.3 Authentication

- **JWT (JSON Web Tokens):** Industry standard
- **API Keys:** For device authentication
- **Why:** Simple, secure, stateless

8.4 Hosting Recommendations

The system can be deployed on:

- AWS (Amazon Web Services)
- Google Cloud Platform
- Heroku
- DigitalOcean
- Any Node.js hosting provider

9 Mobile Integration Readiness

9.1 What Mobile Developers Can Do Now

Your mobile app team can immediately start building:

1. **Login Screen**

- User authentication
- Token generation and storage

2. **Dashboard**
    - Display all owned devices
    - Show battery levels
    - Show last connection time
3. **Device Claiming**
    - QR code scanner
    - Manual device ID entry
    - Device naming
4. **Adherence Tracking**
    - Calendar view of medication events
    - Daily/weekly/monthly summaries
    - Missed dose alerts
5. **Caregiver Features**
    - Assign caregivers to devices
    - Caregiver dashboard view
    - Remove caregiver access

9.2 Example Integration Code Provided

We've provided:

- cURL examples for every API
- Postman collection (import and test immediately)
- Sample payloads with real data
- Authentication flow examples

10 Testing & Quality Assurance

10.1 What's Been Tested

- All API endpoints verified working
- Authentication and authorization tested
- Duplicate event prevention confirmed
- Multi-device scenarios validated
- Caregiver access controls verified
- Error handling tested

10.2 Test Tools Provided

- Token generation utility
- Sample test data
- API testing scripts

11 Next Steps

11.1 Immediate Actions

1. **Review Documentation:** Go through the provided API documentation
2. **Test APIs:** Use Postman collection to test all endpoints
3. **Mobile Integration:** Share API docs with mobile development team
4. **Deploy to Production:** Follow deployment guide to launch

11.2 Future Enhancements (Optional)

If you want to expand the system later:

- Add SMS/email notifications for missed doses
- Implement OTA firmware updates
- Add analytics dashboard
- Integration with health records systems
- Multi-language support

12 Support & Maintenance

12.1 Code Quality

- Clean, well-commented code
- Follows industry best practices
- Easy for new developers to understand
- Modular design for easy updates

12.2 Documentation Maintained

- All code documented
- API documentation up-to-date
- Architecture diagrams included
- Troubleshooting guides provided

13 Conclusion

Project Status: Complete

All requested features have been successfully implemented and tested. The system is production-ready and waiting for mobile app integration.

Key Achievements:

- Secure, scalable backend system
- Complete API suite for mobile apps
- Multi-device and caregiver support
- Comprehensive documentation
- Production-ready code
- Easy handoff to mobile team

No Follow-up Required: Your mobile development team has everything they need to build the app without requiring additional backend support.

14 Appendix: Quick Reference

14.1 Key URLs

- **API Base URL:** `http://localhost:5000` (development)
- **User APIs:** `/api/user/*`
- **Caregiver APIs:** `/api/caregiver/*`
- **Device APIs:** `/api/ingestion/*`

14.2 Authentication Headers

- **User/Caregiver:** `Authorization: Bearer <JWT_TOKEN>`
- **Devices:** `x-api-key: <DEVICE_API_KEY>`

14.3 Key Files Location

- **API Docs:** `docs/API_REFERENCE.md`
- **Architecture:** `docs/ARCHITECTURE.md`
- **Database Schema:** `docs/DATABASE_SCHEMA.md`
- **Deployment:** `docs/DEPLOYMENT.md`